Proceso de Desarrollo: Juego Pokémon (Fuego-Agua-Planta) | José Ángel Alejo Sillero

Índice

- Proceso de Desarrollo: Juego Pokémon (Fuego-Agua-Planta) | José Ángel Alejo Sillero
  - Índice
  - **Introducción**
  - **Proyecto Original**
    - **Características del Proyecto Base**
    - **Estructura Original**
  - **Transformación Realizada**
    - **Cambios Conceptuales**
      - **1. Tema del Juego**
      - **2. Lógica de Ventajas**
      - **3. Nombres de Archivos**
  - **Archivos Creados/Modificados**
    - **1. JFramePokemon.java (Adaptado)**
    - **2. JFramePokemon.form (Archivo XML de NetBeans)**
    - **3. ClassVentajas.java (Adaptado)**
      - **Lógica de Victoria Adaptada**
    - **4. PokemonIconGenerator.java (NUEVO)**
      - **Icono de Agua**
      - **Icono de Fuego**
      - **Icono de Planta**
    - **5. TriangleAdvantagePanel.java (NUEVO)**
  - **Implementaciones Técnicas Detalladas**
    - **Sistema de Iconos Deshabilitados**
    - **GroupLayout y Alineación de Componentes**
    - **Ajustes Específicos de Dimensiones**
  - **Problemas Encontrados y Soluciones**
    - **1. Iconos de Botones Deshabilitados Grises**
    - **2. TextAreas Expandiéndose Infinitamente**
    - **3. TrianglePanel No Carga en NetBeans**
    - **4. Desalineación de Botones**
    - **5. Texto "Reinicio Total" Recortado**
    - **6. Fondo del TrianglePanel con Color**
    - **7. Triángulo Desaparecido tras Cambios Manuales**
  - **Estado Final del Proyecto**
    - **Estructura de Archivos**
    - **Características Implementadas**
    - **Compilación y Ejecución**
    - **Métricas del Proyecto**
  - **Capturas Conceptuales del Layout**
    - **Distribución de Componentes**

- **Lecciones Aprendidas**
- **Referencias**
- **Notas Finales**

Introducción

Este documento describe el proceso completo de transformación de un juego de Piedra-Papel-Tijera a un juego temático de Pokémon basado en el triángulo de ventajas de tipos: Fuego > Planta > Agua > Fuego.

- **Autor:** José Ángel
- **Fecha:** Diciembre 2025
- **Herramientas:** NetBeans IDE, Java Swing, Maven, VSCode (modificar propiedades a las que NetBeans no da acceso fácilmente y crear documentación Markdown)
- **JDK:** Java SE 21

Proyecto Original

Características del Proyecto Base

- Nombre: Piedra-Papel-Tijera (PPT.java)
- Mecánica: Juego clásico con lógica de ventajas: Piedra > Tijera > Papel > Piedra
- Interfaz: Java Swing con NetBeans GUI Builder
- Componentes:
  - 3 botones de usuario (Piedra, Papel, Tijera)
  - 3 botones de máquina (deshabilitados)
  - 2 TextAreas (puntuación y rondas)
  - 1 JLabel (resultado)
  - 2 botones de reinicio

Estructura Original

```
├── PPT.java (JFrame con GUI)
└── ClassVentajas.java (lógica del juego)
```

Transformación Realizada

Cambios Conceptuales

1. Tema del Juego

- Antes: Piedra-Papel-Tijera
- Después: Tipos Pokémon (Fuego-Agua-Planta)

2. Lógica de Ventajas

```
// Antes (PPT)
Piedra > Tijera
Tijera > Papel
Papel > Piedra

// Después (Pokémon)
Agua > Fuego
Fuego > Planta
Planta > Agua
```

3. Nombres de Archivos

- PPT.java → JFramePokemon.java
- ClassVentajas.java → Adaptado para lógica Pokémon

Archivos Creados/Modificados

1. JFramePokemon.java (Adaptado)

Ruta: `src/main/java/com/mycompany/practicaswing/JFramePokemon.java`

Cambios principales:

- Renombrado de clase de `PPT` a `JFramePokemon`
- Textos de botones: "Agua", "Fuego", "Planta"
- Integración de sistema de iconos
- Agregado de panel triangular de ventajas
- Configuración de iconos deshabilitados para botones de máquina

Métodos clave agregados:

```
private void configurarIconos() {
    // Crea y asigna iconos de tipos Pokémon a botones
    javax.swing.ImageIcon iconoAgua = PokemonIconGenerator.createWaterIcon();
    javax.swing.ImageIcon iconoFuego = PokemonIconGenerator.createFireIcon();
    javax.swing.ImageIcon iconoPlanta = PokemonIconGenerator.createGrassIcon();

    // Asignación a botones del jugador
    jButton1.setIcon(iconoAgua);
    jButton2.setIcon(iconoFuego);
    jButton3.setIcon(iconoPlanta);

    // Configuración especial para botones deshabilitados (máquina)
    jButton4.setDisabledIcon(iconoAgua);
    jButton5.setDisabledIcon(iconoFuego);
    jButton6.setDisabledIcon(iconoPlanta);
}

private void configurarPosicionTexto(javax.swing.JButton boton) {
    // Coloca texto debajo del icono
    boton.setHorizontalTextPosition(javax.swing.SwingConstants.CENTER);
    boton.setVerticalTextPosition(javax.swing.SwingConstants.BOTTOM);
    boton.setIconTextGap(5);
}
```

2. JFramePokemon.form (Archivo XML de NetBeans)

Ruta: `src/main/java/com/mycompany/practicaswing/JFramePokemon.form`

Configuración del Layout:

- Uso de GroupLayout para alineación precisa
- Componente personalizado trianglePanel integrado
- Ajustes de espaciado y dimensiones

Configuración clave del trianglePanel:

```
<Component class="com.mycompany.practicaswing.TriangleAdvantagePanel"
name="trianglePanel">
  <AuxValues>
    <AuxValue name="JavaCodeGenerator_CreateCodeCustom"
              type="java.lang.String"
              value="new TriangleAdvantagePanel()"/>
  </AuxValues>
</Component>
```

3. ClassVentajas.java (Adaptado)

Ruta: `src/main/java/com/mycompany/practicaswing/ClassVentajas.java`

Cambios en la lógica:

Lógica de Victoria Adaptada

```
// ANTES (Piedra-Papel-Tijera)
if ((boton.getText().equals("Piedra") && allBtn[NumRandom].getText().equals("Tijera"))
||
    (boton.getText().equals("Tijera") && allBtn[NumRandom].getText().equals("Papel"))
||
    (boton.getText().equals("Papel") && allBtn[NumRandom].getText().equals("Piedra")))
{
    // Victoria
}

// DESPUÉS (Pokémon)
if ((boton.getText().equals("Agua") && allBtn[NumRandom].getText().equals("Fuego")) ||
    (boton.getText().equals("Planta") && allBtn[NumRandom].getText().equals("Agua"))
||
    (boton.getText().equals("Fuego") && allBtn[NumRandom].getText().equals("Planta")))
{
    // Victoria: Agua>Fuego, Planta>Agua, Fuego>Planta
    JFramePokemon.jLabel1.setText("¡Has ganado!");
    JFramePokemon.jLabel1.setBackground(Color.green);
    jugador++;
}
```

Comentarios de documentación mejorados:

```
/** Reglas del juego:
 * - Fuego gana a Planta
 * - Planta gana a Agua
 * - Agua gana a Fuego
 * - Si ambos eligen lo mismo: empate
 */
```

4. PokemonIconGenerator.java (NUEVO)

Ruta: `src/main/java/com/mycompany/practicaswing/PokemonIconGenerator.java`

Propósito: Generación programática de iconos para tipos Pokémon usando Graphics2D.

Iconos implementados:

Icono de Agua

```
public static ImageIcon createWaterIcon() {
    BufferedImage img = new BufferedImage(48, 48, BufferedImage.TYPE_INT_ARGB);
    Graphics2D g2d = img.createGraphics();
    g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING, VALUE_ANTIALIAS_ON);

    // Fondo circular azul claro
    g2d.setColor(new Color(100, 180, 255, 200));
    g2d.fillOval(2, 2, 44, 44);

    // Forma de gota de agua
    int[] xPoints = {24, 16, 14, 24, 34, 32};
    int[] yPoints = {12, 20, 30, 38, 30, 20};
    g2d.setColor(new Color(30, 100, 200));
    g2d.fillPolygon(xPoints, yPoints, 6);

    // Brillo en la gota
    g2d.setColor(new Color(150, 220, 255, 180));
    g2d.fillOval(21, 18, 5, 5);

    g2d.dispose();
    return new ImageIcon(img);
}
```

Icono de Fuego

```
public static ImageIcon createFireIcon() {
    // Llama externa (rojo-naranja)
    g2d.setColor(new Color(255, 100, 30));
    // Forma de llama usando polígonos de 8 puntos

    // Llama interna (amarillo)
    g2d.setColor(new Color(255, 220, 50));
    // Polígono más pequeño para efecto de profundidad
}
```

Icono de Planta

```
public static ImageIcon createGrassIcon() {
    // Forma de hoja con polígono de 8 puntos
    g2d.setColor(new Color(50, 180, 50));
    g2d.fillPolygon(xPoints, yPoints, 8);

    // Nervio central
    g2d.setColor(new Color(80, 200, 80));
    g2d.fillRect(23, 14, 2, 22);

    // Nervios laterales usando drawLine
}
```

Características técnicas:

- Tamaño: 48x48 píxeles
- Antialiasing activado para bordes suaves
- Transparencia ARGB para fondos
- Colores temáticos según tipo

5. TriangleAdvantagePanel.java (NUEVO)

Ruta: `src/main/java/com/mycompany/practicaswing/TriangleAdvantagePanel.java`

Propósito: Panel personalizado que dibuja un triángulo equilátero mostrando las relaciones de ventaja entre tipos.

Implementación:

```
public class TriangleAdvantagePanel extends JPanel {
    private static final int PANEL_WIDTH = 180;
    private static final int PANEL_HEIGHT = 160;
    private static final int ICON_SIZE = 48;

    public TriangleAdvantagePanel() {
        setPreferredSize(new Dimension(PANEL_WIDTH, PANEL_HEIGHT));
        setOpaque(false); // Fondo transparente para integración
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2d = (Graphics2D) g;
        g2d.setRenderingHint(KEY_ANTIALIASING, VALUE_ANTIALIAS_ON);

        // Cálculo de coordenadas de triángulo equilátero
        int centerX = PANEL_WIDTH / 2;
        int topY = 20;
        int bottomY = PANEL_HEIGHT - 30;
        int leftX = 40;
        int rightX = PANEL_WIDTH - 40;

        // Punto superior: Agua (x1, y1)
        // Punto inferior izquierdo: Fuego (x2, y2)
```

```
        // Punto inferior derecho: Planta (x3, y3)

        // Dibujar triángulo con líneas negras de grosor 2
        g2d.setColor(Color.BLACK);
        g2d.setStroke(new BasicStroke(2));
        g2d.drawLine(x1, y1, x2, y2);
        g2d.drawLine(x2, y2, x3, y3);
        g2d.drawLine(x3, y3, x1, y1);

        // Colocar iconos en vértices
        PokemonIconGenerator.createWaterIcon().paintIcon(this, g2d,
            x1 - ICON_SIZE/2, y1 - ICON_SIZE/2);
        PokemonIconGenerator.createFireIcon().paintIcon(this, g2d,
            x2 - ICON_SIZE/2, y2 - ICON_SIZE/2 + 10);
        PokemonIconGenerator.createGrassIcon().paintIcon(this, g2d,
            x3 - ICON_SIZE/2, y3 - ICON_SIZE/2 + 10);

        // Dibujar símbolos ">" en los lados
        g2d.setFont(new Font("Arial", Font.BOLD, 16));
        g2d.setColor(new Color(0, 100, 0)); // Verde oscuro

        // Agua > Fuego (lado izquierdo)
        g2d.drawString(">", (x1 + x2) / 2 - 15, (y1 + y2) / 2);

        // Fuego > Planta (lado inferior)
        g2d.drawString(">", (x2 + x3) / 2 - 5, (y2 + y3) / 2 + 15);

        // Planta > Agua (lado derecho)
        g2d.drawString(">", (x3 + x1) / 2 + 5, (y3 + y1) / 2);
    }
}
```

Características:

- Fondo transparente (setOpaque(false))
- Triángulo equilátero perfecto
- Iconos centrados en vértices
- Símbolos ">" indicando dirección de ventaja
- Integración con sistema de iconos

Implementaciones Técnicas Detalladas

Sistema de Iconos Deshabilitados

Problema inicial: Los botones de la máquina (deshabilitados) mostraban iconos grises en lugar de coloreados.

Solución implementada:

```
jButton4.setIcon(iconoAgua);
jButton4.setDisabledIcon(iconoAgua); // Mantiene icono coloreado cuando disabled
jButton5.setIcon(iconoFuego);
jButton5.setDisabledIcon(iconoFuego);
jButton6.setIcon(iconoPlanta);
jButton6.setDisabledIcon(iconoPlanta);
```

GroupLayout y Alineación de Componentes

Desafío: Alinear correctamente botones, TextAreas y el panel triangular.

Solución - Layout Horizontal:

```
jPanel1Layout.setHorizontalGroup(
    jPanel1Layout.createParallelGroup(Alignment.LEADING)
    .addGroup(jPanel1Layout.createSequentialGroup()
        .addGap(32) // Margen izquierdo
        // Grupo de botones usuario + jButton8 (Reiniciar Puntuación)
        .addGroup(jPanel1Layout.createParallelGroup(Alignment.LEADING)
            .addComponent(jButton1, 80, 80, 80)
            .addComponent(jButton2, 80, 80, 80)
            .addComponent(jButton3, 80, 80, 80))
        .addGap(35) // Separación
        // Panel triangular
        .addComponent(trianglePanel, PREFERRED_SIZE, DEFAULT_SIZE, PREFERRED_SIZE)
        .addGap(18)
        // Grupo de botones máquina + jButton7 (Reinicio Total)
        .addGroup(jPanel1Layout.createParallelGroup(Alignment.LEADING)
            .addComponent(jButton4, 80, 80, 80)
            .addComponent(jButton5, 80, 80, 80)
            .addComponent(jButton6, 80, 80, 80)
            .addComponent(jButton7, 120, 120, 120)) // Más ancho para texto completo
        .addContainerGap())
);
```

Solución - Layout Vertical:

```
jPanel1Layout.setVerticalGroup(
    jPanel1Layout.createParallelGroup(Alignment.LEADING)
    .addGroup(jPanel1Layout.createSequentialGroup()
```

```
        .addContainerGap()
        // Fila 1: Botones usuario (izq) + TrianglePanel (centro) + Botones máquina
(der)
        .addGroup(jPanel1Layout.createParallelGroup(Alignment.TRAILING)
            // Columna izquierda (usuario)
            .addGroup(jPanel1Layout.createSequentialGroup()
                .addComponent(jButton1, 70, 70, 70)
                .addGap(44)
                .addComponent(jButton2, 70, 70, 70)
                .addGap(44)
                .addComponent(jButton3, 70, 70, 70))
            // Columna central (info)
            .addGroup(jPanel1Layout.createSequentialGroup()
                .addComponent(trianglePanel, 172, 172, 172)
                .addGap(6)
                .addComponent(jScrollPane1, PREFERRED_SIZE)
                .addGap(18)
                .addComponent(jScrollPane2, PREFERRED_SIZE)
                .addGap(18)
                .addComponent(jLabel1))
            // Columna derecha (máquina)
            .addGroup(jPanel1Layout.createSequentialGroup()
                .addComponent(jButton4, 70, 70, 70)
                .addGap(44)
                .addComponent(jButton5, 70, 70, 70)
                .addGap(44)
                .addComponent(jButton6, 70, 70, 70)))
        // Fila 2: Botones de reinicio (baseline alignment)
        .addGroup(jPanel1Layout.createParallelGroup(Alignment.BASELINE)
            .addComponent(jButton7)
            .addComponent(jButton8))
        .addContainerGap(18, Short.MAX_VALUE))
);
```

Ajustes Específicos de Dimensiones

TextAreas: Ancho fijo de 97px para evitar expansión descontrolada

```
<Component id="jScrollPane1" min="-2" pref="97" max="-2" attributes="0"/>
<Component id="jScrollPane2" min="-2" pref="97" max="-2" attributes="0"/>
```

Botón Reinicio Total: 120px de ancho para texto completo

```
jButton7.setPreferredSize(new Dimension(120, defaultHeight));
```

Panel Triangular: 180x160px con preferredSize fijo

```
setPreferredSize(new Dimension(180, 160));
```

Problemas Encontrados y Soluciones

1. Iconos de Botones Deshabilitados Grises

Problema: Los botones de la máquina (disabled) mostraban los iconos en gris.

Causa: Por defecto, Swing aplica un filtro gris a iconos de componentes deshabilitados.

Solución:

```
jButton4.setDisabledIcon(iconoAgua); // Fuerza icono original incluso disabled
```

2. TextAreas Expandiéndose Infinitamente

Problema: Los JScrollPane con TextAreas crecían horizontalmente sin límite.

Causa: Configuración de layout con max="32767" (Short.MAX_VALUE).

Solución:

```
<!-- Antes -->
<Component id="jScrollPane1" pref="0" max="32767" attributes="0"/>

<!-- Después -->
<Component id="jScrollPane1" min="-2" pref="97" max="-2" attributes="0"/>
```

3. TrianglePanel No Carga en NetBeans

Problema: Error al abrir JFramePokemon.form en NetBeans:

```
Cannot load component class: TriangleAdvantagePanel
```

Causa: Referencia en .form sin el nombre completo del paquete.

Solución:

```
<!-- Antes -->
<Component class="TriangleAdvantagePanel" name="trianglePanel">

<!-- Después -->
<Component class="com.mycompany.practicaswing.TriangleAdvantagePanel"
name="trianglePanel">
```

4. Desalineación de Botones

Problema: jButton7 y jButton8 no alineados con sus columnas respectivas.

Causa: Agrupación incorrecta en GroupLayout horizontal.

Solución:

```
// Mover jButton8 al grupo de botones de usuario
.addGroup(jPanel1Layout.createParallelGroup(Alignment.LEADING)
    .addComponent(jButton1, 80, 80, 80)
    .addComponent(jButton2, 80, 80, 80)
    .addComponent(jButton3, 80, 80, 80)
    // jButton8 ya no está en grupo separado)

// Mover jButton7 al grupo de botones de máquina
.addGroup(jPanel1Layout.createParallelGroup(Alignment.LEADING)
    .addComponent(jButton4, 80, 80, 80)
    .addComponent(jButton5, 80, 80, 80)
    .addComponent(jButton6, 80, 80, 80)
    .addComponent(jButton7, 120, 120, 120)) // Alineado verticalmente
```

5. Texto "Reinicio Total" Recortado

Problema: El texto del jButton7 aparecía como "Reinic..."

Causa: Ancho de botón de 80px insuficiente.

Solución:

```
// Aumentar ancho específico de jButton7
.addComponent(jButton7, 120, 120, 120) // De 80 a 120 píxeles
```

6. Fondo del TrianglePanel con Color

Problema: Panel triangular tenía fondo blanco/coloreado en lugar de transparente.

Causa: setOpaque(true) por defecto en JPanel.

Solución:

```
public TriangleAdvantagePanel() {
    setPreferredSize(new Dimension(PANEL_WIDTH, PANEL_HEIGHT));
    setOpaque(false); // Transparencia para integrarse con jPanel1
}
```

7. Triángulo Desaparecido tras Cambios Manuales

Problema: Al hacer cambios en NetBeans, el trianglePanel se eliminó del layout.

Causa: Usuario deshizo cambios o NetBeans regeneró el código.

Solución: Re-agregar manualmente en ambos archivos:

En .form:

```
<Component class="com.mycompany.practicaswing.TriangleAdvantagePanel"
name="trianglePanel">
  <AuxValues>
    <AuxValue name="JavaCodeGenerator_CreateCodeCustom"
              type="java.lang.String"
              value="new TriangleAdvantagePanel()"/>
  </AuxValues>
</Component>
```

En .java:

```
// Inicialización
trianglePanel = new TriangleAdvantagePanel();

// Declaración
private com.mycompany.practicaswing.TriangleAdvantagePanel trianglePanel;
```

Estado Final del Proyecto

Estructura de Archivos

```
PracticaSwing/
├── src/main/java/com/mycompany/practicaswing/
│   ├── ClassVentajas.java           (Lógica del juego)
│   ├── JFramePokemon.java           (Interfaz gráfica)
│   ├── JFramePokemon.form           (Diseño NetBeans)
│   ├── PokemonIconGenerator.java    (Generador de iconos)
│   └── TriangleAdvantagePanel.java  (Panel triángulo ventajas)
├── pom.xml                          (Configuración Maven)
└── PROCESO_DESARROLLO_POKEMON.md    (Este documento)
```

Características Implementadas

Interfaz Gráfica:

- 3 botones de usuario con iconos Pokémon (Agua, Fuego, Planta)
- 3 botones de máquina (deshabilitados) con iconos coloreados
- Panel triangular mostrando relaciones de ventaja
- 2 TextAreas para puntuación y rondas
- 1 JLabel para resultado
- 2 botones de reinicio (parcial y total)

Lógica de Juego:

- Triángulo de ventajas Pokémon: Agua > Fuego > Planta > Agua
- Sistema de puntuación (jugador, máquina, empates)
- Sistema de rondas (total, ganadas, perdidas, empatadas)
- Reinicio parcial (mantiene estadísticas de rondas)
- Reinicio total (resetea todo)
- Selección visual del usuario (botón gris persistente)

Elementos Visuales:

- Iconos generados programáticamente (48x48px)
- Panel triangular con geometría equilátero perfecto
- Antialiasing para gráficos suaves
- Colores temáticos según tipo Pokémon
- Layout responsive con GroupLayout

Compilación y Ejecución

Compilar:

```
mvn compile
```

Ejecutar:

```
java -cp target/classes com.mycompany.practicaswing.ClassVentajas
```

O desde NetBeans: **Run > Run Project (F6)**

Crear JAR ejecutable:

Para generar un archivo JAR ejecutable del proyecto, navega al directorio `target/classes` y ejecuta:

```
jar cvfe JuegoPokemon.jar com.mycompany.practicaswing.ClassVentajas
com/mycompany/practicaswing/*.class
```

Este comando crea `JuegoPokemon.jar` con ClassVentajas como clase principal, incluyendo todas las clases necesarias (JFramePokemon, PokemonIconGenerator, TriangleAdvantagePanel).

Para ejecutar el JAR:

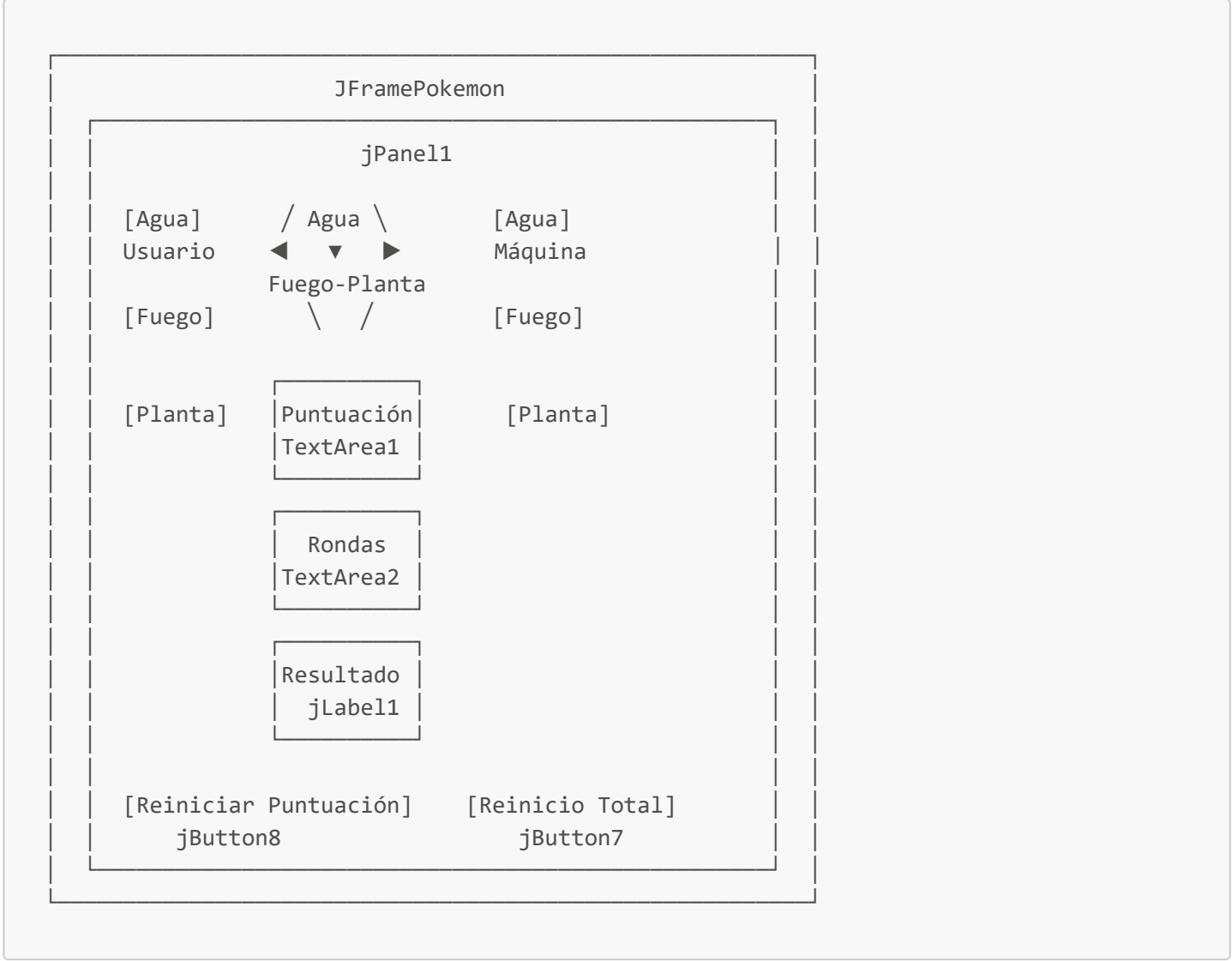
```
java -jar JuegoPokemon.jar
```

Métricas del Proyecto

Archivo	Líneas de Código	Propósito
ClassVentajas.java	161	Lógica del juego
JFramePokemon.java	323	Interfaz gráfica
JFramePokemon.form	320 (XML)	Diseño NetBeans
PokemonIconGenerator.java	118	Generación de iconos
TriangleAdvantagePanel.java	89	Panel triángulo
TOTAL	1011	

Capturas Conceptuales del Layout

Distribución de Componentes



Lecciones Aprendidas

1. GroupLayout es poderoso pero complejo: Requiere entender jerarquías de grupos paralelos y secuenciales.
2. NetBeans GUI Builder: Facilita diseño visual, pero modificaciones manuales requieren sincronización .form ↔ .java.
3. Componentes personalizados en NetBeans: Necesitan nombre completo de paquete en archivo .form.
4. Graphics2D: Permite crear gráficos complejos sin dependencias externas de imágenes.
5. Disabled icons: Requieren configuración explícita con setDisabledIcon() para evitar filtros grises.
6. Dimensiones fijas vs. flexibles: Componentes como TextAreas necesitan restricciones de tamaño para evitar expansión descontrolada.
7. Transparencia en paneles: setOpaque(false) es esencial para integrar componentes custom con fondos coloreados.

Referencias

- Java Swing Documentation
- GroupLayout Tutorial
- Graphics2D API
- NetBeans GUI Builder Guide

Notas Finales

Este proyecto demuestra la transformación completa de un juego clásico a una temática moderna, implementando los siguientes conceptos:

- Programación orientada a objetos (clases separadas para lógica y UI)
- Gráficos 2D personalizados (iconos y panel triangular)
- Diseño de interfaces con GroupLayout
- Documentación exhaustiva con comentarios Javadoc
- Resolución de problemas a través de iteración y debugging

El resultado es una aplicación funcional, visualmente atractiva y educativa que mantiene la mecánica del juego original mientras incorpora elementos temáticos de Pokémon.

- **Autor:** José Ángel
- **Fecha de finalización:** 20 de diciembre de 2025
- **Versión del documento:** 2.0